

Deliverable 3: Design Phase

System Architecture

A. Presentation Layer:

This layer is user interface layer based on the wireframe screens.

Components from the wireframe:

- **Patient Screens:**
 - Registration.
 - Login.
 - View Appointments.
 - Book Appointment.
 - Modify Appointment.
 - Cancel Appointment.
 - Submit Patient Info.
- **Doctor Screens:**
 - Doctor Dashboard.
 - Patient History.
- **Receptionist Screens:**
 - Receptionist Dashboard.
 - Manage Appointments.
 - Register Patient.
- **Admin Screen:**
 - Admin Dashboard.
- **AI Result Screens:**
 - AI Analysis.
 - Urgency Score.
 - Priority Queue.
 - Time Slot Recommendation.
 - No-Show Prediction.
 - Reminder Sent.
 - Waitlist.

Purpose:

- Collect User Input.
- Displays Appointment Results.
- Show AI Recommendations.
- Allows Staff to manage Appointments.

B. Application Layer:

This Layer Controls System Flow.

Main Components:

- Authentication Controller.
- Patient Controller.
- Appointment Controller.
- Prioritization Controller.
- Scheduling Controller.
- Notification Controller.
- Dashboard Controller.

Purpose:

- Receives requests from the UI.
- Validates and routes them.
- Connects user actions with AI and database functions.

C. Business / AI Logic Layer:

This is the core logic Layer.

Main Components:

- Patient Triage Service.
- AI Prioritization Engine.
- Urgency Scoring Module.
- Time Slot Recommendation Engine.
- No-Show Prediction Module.
- Waitlist Management Service.
- Appointment Rules Engine.

Purpose:

- Analyzes Patient Symptoms and Vitals.
- Calculates Urgency.
- Prioritizes Appointments.
- Suggests the best slot.
- Predicts missed Appointments.
- Manages waitlist behavior.

D. Data Access Layer:

This layer handles communication between system logic and storage.

Main Components:

- Patient Repository.
- Appointment Repository.
- Priority Score Repository.
- Waitlist Repository.
- Reminder Repository.
- History Repository.
- Analytics Repository.

Purpose:

- Saves and retrieves data.
- Separates Business Logic Database details.

E. Database / Infrastructure Layer:

This is the storage and support services layer.

Main Components:

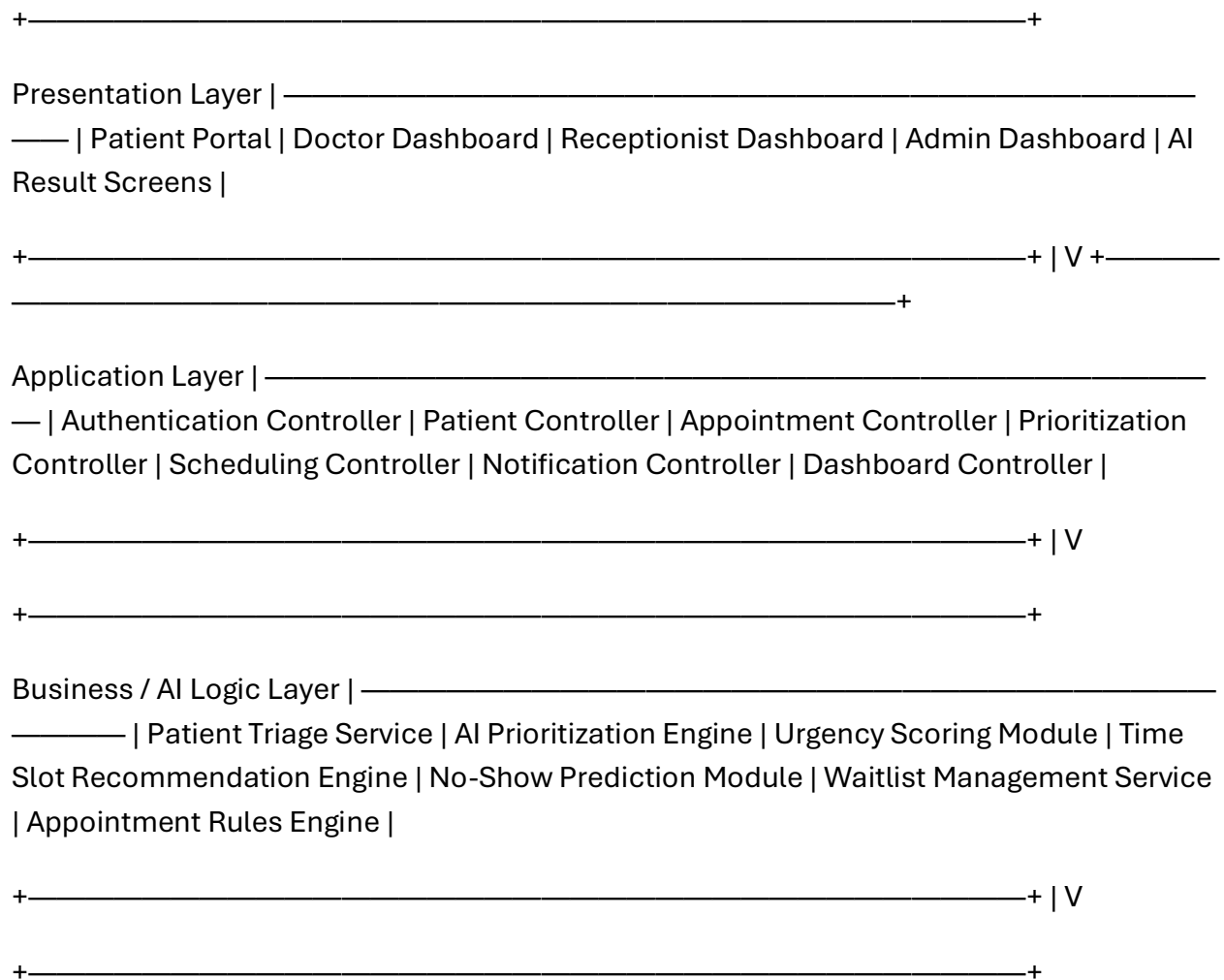
- Patient Database.
- Appointment Database.
- Doctor Schedule Database.
- AI Results Database.

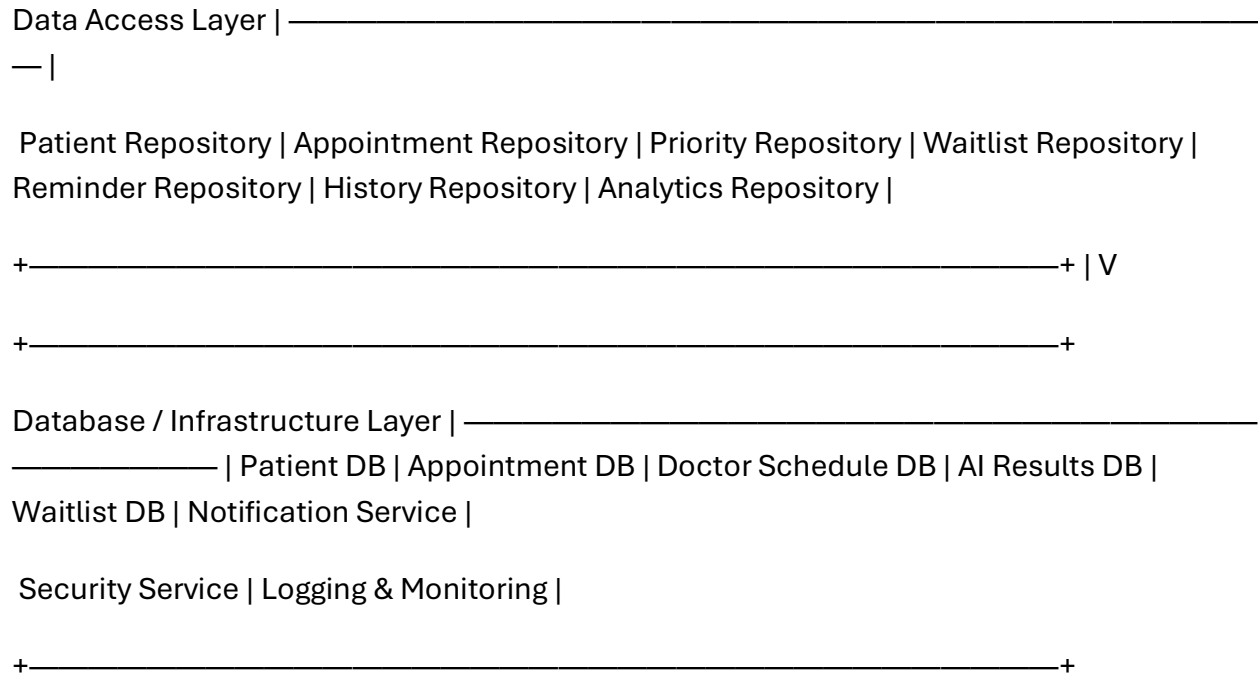
- Waitlist Database.
- Notification Service.
- Security / Authentication Service.
- Logging / Monitoring Service.

Purpose:

- Stores all persistent data.
- Supports SMS/email reminders.
- Protect access.
- Records system activity.

Architecture Diagram:





Data Flow based on wireframe:

Here is the main data flow from the screens.

Flow 1: Patient registration and login. From **P1 Registration** and **P2 Login**, the patient enters username, password, and email. The Presentation Layer sends this data to the **Authentication Controller** in the Application Layer, which validates it and stores or checks it through the Data Access Layer and the user database.

Flow 2: Patient submits health information. From **P7 Submit Patient Info**, the patient enters name, age, symptoms, pain level, condition, and vitals. This goes from the Presentation Layer to the **Patient Controller**, then to the **Patient Triage Service** and **AI Prioritization Engine** in the Business Layer for analysis. The results are saved in the database.

Flow 3: AI Analyzes urgency. From **S1 AI Analysis** and **S2 Urgency Score**, the system processes the patient input, runs the prioritization model, and produces an urgency score and level such as high, medium, or low. This is handled mainly in the Business / AI Logic Layer, then shown back on the screen through the Application Layer.

Flow 4: Appointment prioritization. From **S3 Appointment Prioritization**, the system ranks patients in a queue using patient condition, urgency, and expected waiting time. This

is produced by the **Urgency Scoring Module**, **Appointment Rules Engine**, and **Waitlist Management Service**.

Flow 5: Recommended time slot. From **S4 Time Slot Recommendation**, the prioritized patient is matched with an available slot based on urgency and availability. This uses the **Time Slot Recommendation Engine**, which checks doctor schedules and appointment availability stored in the database.

Flow 6: Booking, Modifying, or Canceling. From **P3–P6**, the patient can view appointments, book one, modify one, or cancel one. These actions go to the **Appointment Controller** and **Scheduling Controller**, which update the appointment records and availability in the database.

Flow 7: No-Show prediction and reminders. From **S5 No-Show Prediction** and **S6 Appointment Reminder**, the system predicts whether a patient may miss the appointment, and if needed sends a reminder through the **Notification Controller** and **Notification Service**.

Flow 8: Staff and doctor review. From **D1 Doctor Dashboard**, **D2 Patient History**, **R1 Receptionist Dashboard**, and **R2 Manage Appointments**, staff can view priority patients, review history, and manage bookings. These dashboards request data through the Application Layer from the repositories in the Data Access Layer.

Flow 9: Admin monitoring. From **A1 Admin Dashboard**, the admin can monitor no-show rate, waiting time, and system usage. These are retrieved from the analytics and reporting components in the lower layers.

Simple Input-process-Output view.

Input:

- Patient registration data.
- Symptoms.
- Pain level.
- Condition.
- Vitals.
- Doctor/Date choice.
- Appointment actions.

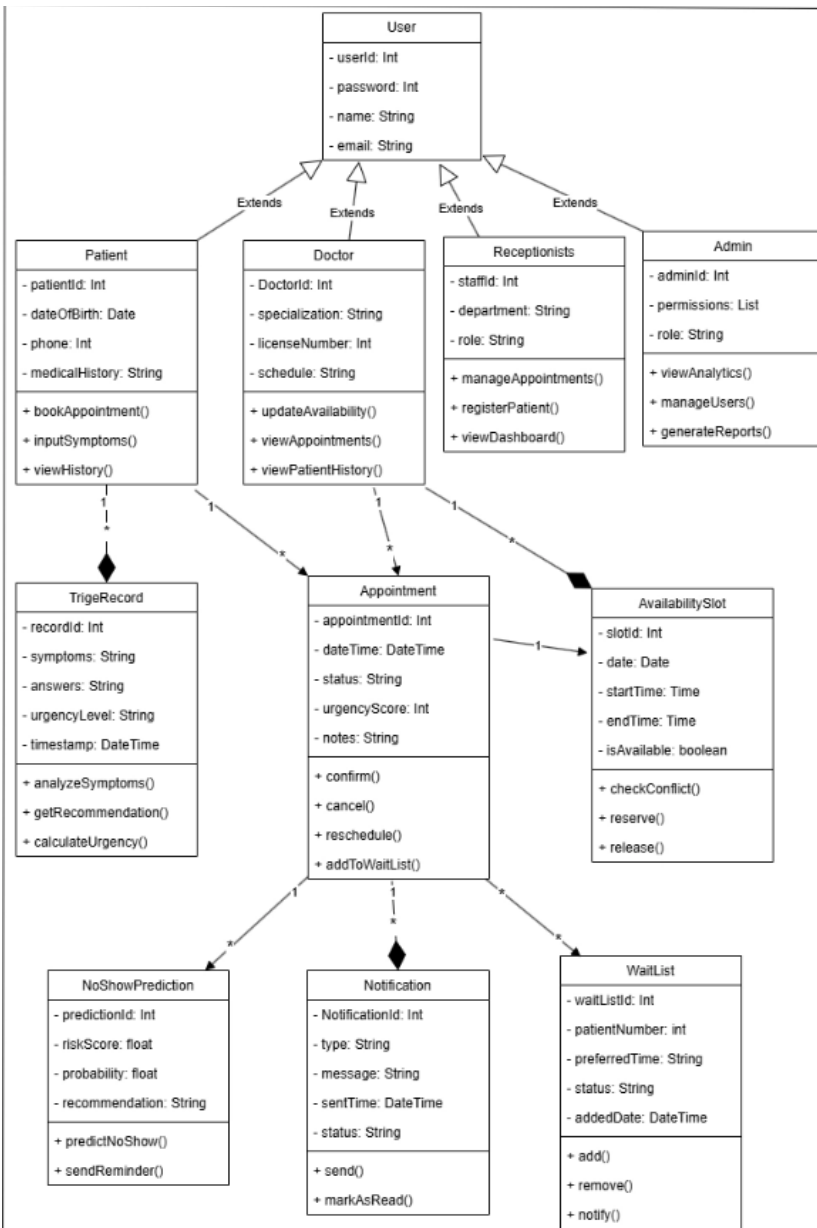
Process:

- Authentication.
- Patient data validation.
- AI urgency analysis.
- Prioritization.
- Slot recommendation.
- No-Show prediction.
- Reminder Generation.

Output:

- Urgency score.
- Priority queue.
- Suggested appointment time.
- Confirmed/Modified/Cancelled appointment.
- Reminders.
- Dashboards and reports.

UML Design Class Diagram



REST API Design

1. System Overview

This REST API allows the patient booking portal, staff dashboards, and external hospital systems to interact with the AI Prioritization engine. It is designed to be lightweight, secure, and focused on the core functional requirements identified in the project plan.

2. Base URL

<https://api.ai-hospital.qa/v1>

3. Authorization & Roles

All API endpoints require authentication using JWT tokens. Authorization is enforced based on user roles.

User Roles:

- Patient → book, view, cancel appointments
- Staff → manage appointments, view dashboards
- Admin → access analytics and system metrics

4. Resources

/patients → patients and user accounts

/appointments → scheduled bookings and time slots

/doctors → medical staff and their availability

/priorities → AI-generated urgency scores

/predictions → no-show risk analysis

/waitlist → pending appointments for full slots

/reminders → automated SMS notifications

/analytics → system performance and utilization metrics

5. API Endpoints & HTTP Methods Used

5.1 - Patients & Appointment Management

Endpoint	Method	Description	Expected Response
/patients/register	POST	Registers a new patient account	201 Created with Patient ID
/appointments	GET	Retrieves available time slots based on doctor and date	200 OK with a list of available slots
/appointments/book	POST	Creates a new appointment. Triggers AI Urgency Score	201 Created with Appointment ID and Urgency Score
/appointments/{id}	PUT	Modifies or cancels an existing appointment	200 OK with update status
/appointments/waitlist	POST	Adds patient to a waitlist if slots are full	201 Created with Waitlist Position

5.2 - AI & Analytics Endpoints

Endpoint	Method	Description	Expected Response
/ai/prioritize	POST	Submits symptoms/health info to calculate urgency	200 OK with urgency_score and recommended_slot
/ai/predict-noshow	GET	Predicts the probability of a patient missing an appointment	200 OK with no_show_probability percentage
/analytics/metrics	GET	Retrieves data for the Admin Dashboard (no-show rates, utilization)	200 OK with JSON hospital KPIs

5.3 - Notifications & Integration

Endpoint	Method	Description	Expected Response
/notifications/send	POST	Triggers automated SMS reminders	202 Accepted
/integration/sync	POST	Pulls/Pushes essential data to existing hospital systems	200 OK Sync status

6. Design Evaluation

Performance:

The API is designed to be lightweight, supporting fast AI responses within 5 seconds, meeting system requirements.

Security:

JWT authentication and role-based access control ensure patient data privacy and secure system access.

Usability:

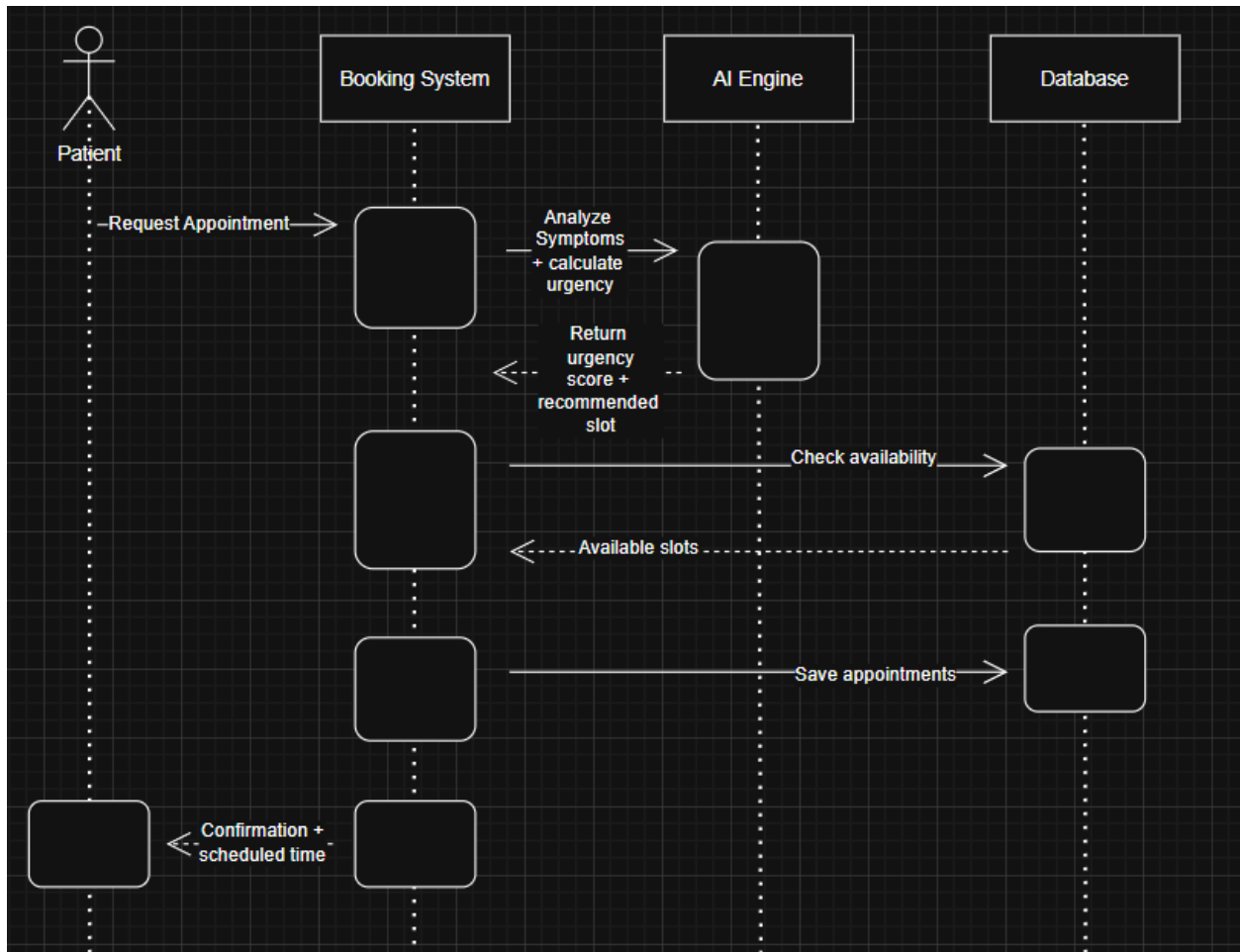
Clear resource structure and RESTful endpoints make the API easy to use and integrate with hospitals systems.

Maintainability:

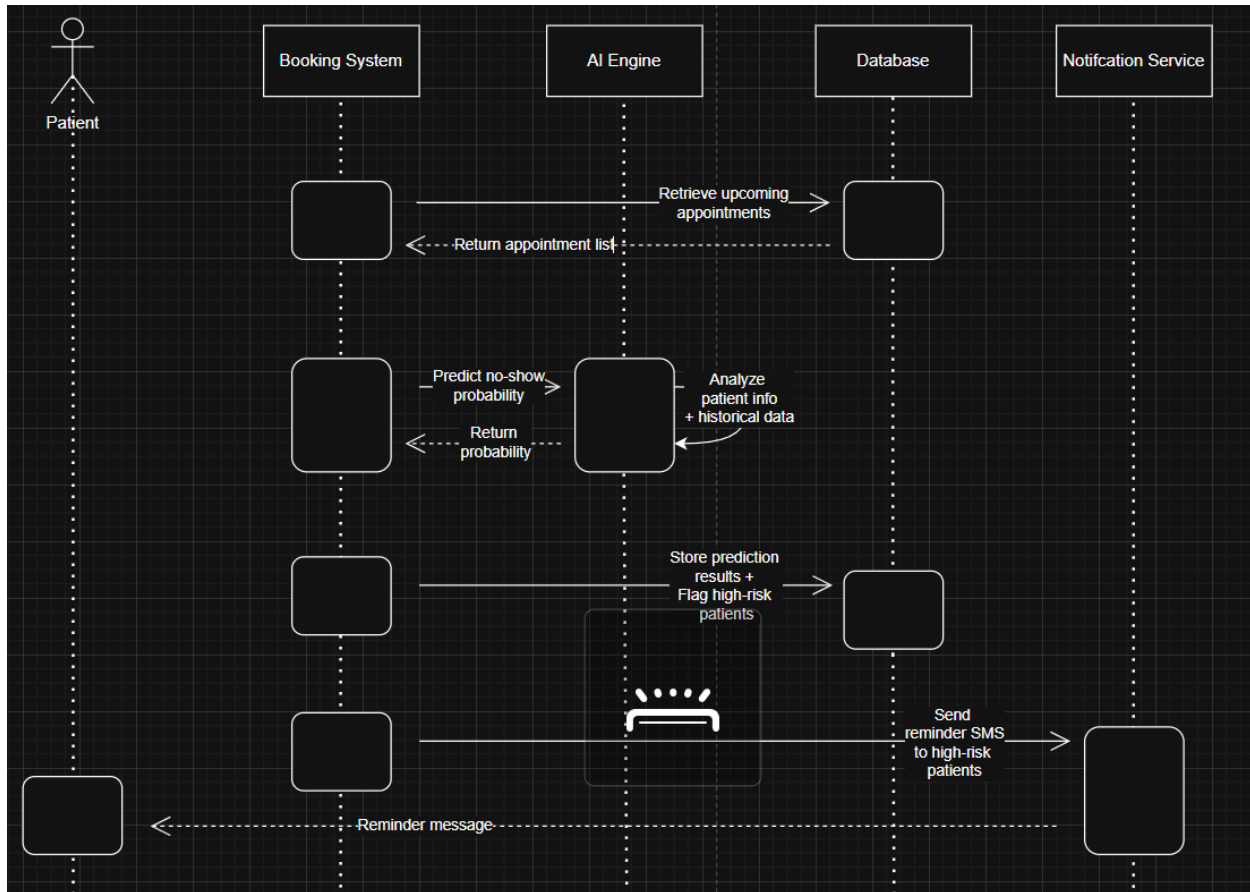
Modular endpoint design (appointments, AI, analytics) allows easy future updates and scalability.

Sequence Diagrams

Sequence Diagram 1



Sequence Diagram 2



Administration Evaluation

What software architecture would you adopt to satisfy these requirements?

To satisfy the identified requirements, I would adopt a layered software architecture. This architecture is appropriate because it separates the system into distinct layers for presentation, application processing, business and AI logic, data access, and infrastructure, which improves maintainability, security, scalability, and clarity of system design.

To what extent does the proposed software architecture satisfy the requirements in terms of performance, security, usability, maintainability?

The proposed layered software architecture satisfies the system requirements to a large extent. It provides good performance by separating processing responsibilities across layers, strong security through controlled access and role-based restrictions, high usability through dedicated user interfaces for each stakeholder, and very good maintainability due

to its modular structure, which allows individual components to be updated or improved without major impact on the rest of the system.

How do you see the delivery and administration of your information system based on user needs?

The delivery and administration of the AI Hospital Appointment Prioritization System are based on the needs of its main users: patients, doctors, receptionists, and administrators. The system should be delivered as a **centralized web-based platform with role-based access**, allowing each user to interact with features relevant to their role.

Patients require a simple interface for booking and managing appointments, doctors need dashboards showing prioritized cases, receptionists need tools for scheduling and queue management, and administrators need access to system monitoring and analytics.

In terms of administration, the system should support **user management, data security, and performance monitoring**. Role-based access ensures privacy, while administrators can oversee system usage and update the AI model as needed.

Overall, the system delivery focuses on usability, while administration ensures security, efficiency, and continuous improvement.